Complex Adaptive Systems:
an Introduction

Franco Zambonelli, Marco Mamei

January 2004, Reggio Emilia

Outline

- What are Systems
  - Components, interactions, dynamics
- What is Adaptivity
  - Openness, situatedness, context-awareness
- What is Complexity
  - Complicated vs. complex Systems
  - Complexity vs. chaos
  - Relations with natural systems
- Why Modern Distributed Systems are CAS?
  - Characteristics of modern distributed systems
  - Needs for CAS and Self-organization
  - Visions of the future
- So What?
  - A rationale for the course organization…

What are Complex Adaptive Systems?

- Let's understand this by characterizing the concepts of
  - ***System***
  - ***Adaptivity***
  - ***Complexity***
- And contextualize this to distributed computational systems

Part 1

- What it a System?

What are “Systems”?

- A System is an ensemble of (situated) individual entities interacting with each other
  - Individual entities:
    - atoms, cells, organs, animals, trees, humans
    - Chips, computers, LANs, sensors robots
  - Interacting
    - Physical forces, protein diffusion, gestures, words
    - Electric Signals, packets, IP datagrams, radio signals
  - Situated
    - Existing in an “environment” (see next)

Modeling Individuals in a System

- In general, an entity in a system is characterized by:
- A state
  - The dimensions fully characterizing the current situation of the individual
    - E.g., position and speed (X,dX/dt) for a physical object
    - E.g., the value of its instance variables for a software object
    - E.g., the “mood” for a human
- Transition rules
  - What events in the system make an individual change its state
    - i.e., what interactions
    - E.g., a gravitational force, the finding of food, a conversation…
- A Behavior
  - What the individual do in the system, how it affect the system
    - E.g., spreading of a gravitational force, eating food, talking, or simply occupy space
    - An object invoking methods of other objects
    - A Servlet changing a database
- A Context (a “position” in the environment)
  - Sometimes, it is useful to model a system and its individual as being “situated” in some environment
    - E.g., a topological space-time universe, a landscape, a room,
    - A servlet context
  - Other times, the environment in turn is modeled in terms of individuals
    - E.g., the space time being made of gravitons, a landscape being made of individual trees, grass, mote, etc.
    - In an OO program, everything is an object

Modeling Interactions in a System

- An interaction is any event in a systems that
  - Caused by an individual in a system
  - Affect that state or one or several individual in the system
- Types of interactions
  - Direct ("Communication")
    - Two particles colliding with each other
    - Two humans talking with each other
    - A client and a server exchanging TCP packets
  - Indirect ("Stigmergic" or mediated by the environment)
    - The modification of the speed of a particles caused by the gravitational field of another particles
    - The spread of pheromones by ants
    - Playing Chess
    - Two servlets accessing the same Attribute
- A note:
  - Sometimes, mediated interactions are not considered (i.e., fields in modern physics are modeled as particles!)
  - But this may be very important in distributed computational systems!!!

Distributed Computational Systems

- A Single computer is a system
  - Many components, to be orchestrated
  - By the operating system
  - Via coordination of access to common resources (stigmergic)
- Computer networks, in general…
  - Individual entities: computers
  - Networks: LAN + Internet
  - Interactions:
    - Direct: Message-passing, Client-server, Events
    - Stigmergic: Access to common databases and resources
- But with a very broad meaning
  - Computers: Supercomputers, PC, PDA, Mobile Phones, embedded computers
  - Networks: Wires, Wireless, GPRS, UMTS, Bluetooth, P2P, The Web, The overlying social networks
  - Interactions: Access to common memory, Access to common artifcats, RF-ID Tags, Broadcasting, Digital Pheromones, Digital fields
  - Will analyse all of these….

Discrete vs. Continuous & Synchronous vs. Asynch. Systems

- The modeling of a system may imply
  - Continuous vs. Discrete state
    - E.g., The position of a particle in the space vs. the position of a client on an IP
  - Continuous vs. Discrete time
    - E.g., The time we perceive vs. the "clock" of a digital computer
    - This affect the way state transitions occur
- Global vs. Local Time
  - Global: all the individual of the system have the same clock, and thus perform state transition at the same time
    - e.g., massed in classical mechanics, synchronized threads on a single computer
    - ***Synchronous***: when they perform state transition at the same time or when there is a sort of global controller for transitions
  - All the individual have individual clocks
    - E.g., the twin paradox of General Relativity, two threads on different computers, a person which is having fun vs. a person which is getting bored
    - ***Asynchronous***: when they do not obey to global clock rules to perform state transitions, and there is not global controller
- Summarizing:
  - **Physics** is discrete in state, continuous in time, and asynchronous
  - **Ecology** is continuous in state, continuous in time, and asynchronous
  - **Computing** is discrete in time, discrete in time and synchronous
  - **Distributed computing** is discrete in time, discrete in time and asynchronous

Local vs. Global System State

- The collection of all states of all individual of a systems (i.e., the **local states**), plus the state of the environment, if needed, determines:
  - The **global state** of the system
- However, in many cases, the global state is represented by more "synthetic" indicators, Because
  - It give a synthetic clue of what is happening in the system
  - Sometimes, it is impossible to determine the global state in terms of all local states
- Examples
  - "The current average speed on the highway is 45mph"
  - "The DISMI network has a throughput of 345kbs"
  - "The unbalance in computer load is 45%"
  - The state of a gas is measured by P & T, because it is impossible to measure the positions and speeds of all its atoms

System Dynamics

- A system evolves in time
  - Via the **local** state transition (local state changes) of its individuals
  - Affecting the **global** system state
- The study of system dynamics imply
- Determining the evolution of the system
  - Given an initial state
  - E.g., will the system reach a global equilibrium? Of what type? From what initial states will reach what equilibrium state? Will it continue evolving indefinitely? What happens when we perturb the system from equilibrium?

The "Phase Space" of a System

- When studying system dynamics, one
  - Does not refer to the evolution in state of all its system components
  - But rather to the evolution of some of its relevant global state indicators
    - E.g., the evolution of a gas is described in terms of how its temperature and pression change in time
  - This is the so called "trajectory in the state space"
    - The dynamic evolution of the system as a point moving in a n-dimensional diagram, starting from an initial point
    - Axis represent the relevant indicators of the global system state
    - And in which time passes as the point moves…
- Typically, the trajectory tend to be "attracted" in specific regions of the diagram
    - "**Attractors**" of the systems (single points for static equilibrium, circles for periodic behaviours, or more complex manifold)
    - **Attractors express stability**: once the system reaches an attractor, it will stay there undefinitely, until perturbed

Part 2

- What is an Adaptive System?

What is Adaptivity?

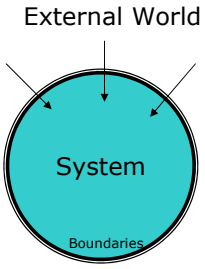
- The capability of a system and/or of its individuals of tuning its behavior to contingent situations
- In general, adaptivity implies that the system is both flexible and robust
  - Re-organizes itself in response to stimuli
  - Preserves the same overall dynamics
  - Preserves (some of) its specific characteristics independently of external contingencies/stimuli
- What are these contingencies requiring adaptivity?
  - Openness → the boundaries of a system are open
  - Environmental Dynamics → the environment may be dynamic and impredictable

Openness

- No system in the world in a closed universe
  - Interactions with the rest of the world necessarily occur
    - E.g., heat flow through a gas container – whatever insulated
    - Gravity of external stars affects the solar system
    - Broadcast IP packets affect any LAN
- So, how can we deal with that?
  - We cannot certainly analyze the thermodynamics of the whole universe to understand how a container of gas behave, not we can calculate the mass of all galaxies and stars to understand the dynamics of the solar system, nor we cannot anslyse the traffic of the whole Internet to configure our LANs…

Dealing with Openness

- Let's "model" explicitly
  - What it is inside the system
  - What it outside
  - So as to define "the boundaries" of the system
- Identify and model the "interactions across boundaries"
  - What energy/heat flow across a gas container
  - What amount of external traffic enters our LAN
- The boundaries of the system somewhat identify the "***environment***" in which the system situated
  - The context in which an individual component situates this include both the other component with which it interacts and the environment
- The environment is an abstraction!!!
- Not always easy to identify



Ignoring Openness

- In some cases, the interactions across the boundaries are so minimal that they can be totally ignored
- The system is "de facto" closed
  - E.g. The movement of major planets around the sun is not perceivably affected by nearby stars
  - E.g., The execution of a specific program in a mono-programmed computer is not perceivably affected by the very limited concurrent activities of the operating system
- Unfortunately, this is not the case for most of real-world natural and computational systems…

Accounting for Openness

- In most cases, the interactions across boundaries are very relevant, for two reasons:
  - **Component Openness.** New individuals may cross boundaries to enter the system, thus changing the very structure of the system
    - E.g., now only heat but also matter can enter a not perfectly closed gas container
    - A bit comet can enter the solar system
    - Students can install new services in the PC of our LAN
  - **Environmental Dynamics**. The environment and its dynamics can perturb the system (e.g. affect the state transitions of individuals or their behavior)
    - A heated gas container will move the internal gas our of equilibrium
    - A nearby black hole will definitely impact of planets movements
    - An external hacker spamming IP packets will dramatically impact on the performances of our internal LAN services
  - The distinction between the two types of openness is clearly conceptual (e.g., the example of gravitons vs. gravitational fields). However, it is greatly practical

Adaptive vs. Non Adaptive Systems

- Adaptivity in response to openness and environmental perturbations/dynamics implies
  - Flexible response: the system adapt its overall dynamics in response to stimuli
  - Robust behavior: at the same time, the system preserve its overall behavior, or at least some of its relevant characteristics
- However, please note that adaptivity is not an absolute concept
  - It depends on what we are interested in in a system
  - It sometimes depends on the eye of the observer and on the level of observation of the system

Earth vs. Heart

- The Earth mechanical dynamics
  - Non adaptive: a perturbation (e.g. a big earthquake) change its rotation period permanently
- The Earth climate
  - Very Adaptive: stable over very long periods despite pollution and sun storms
  - But non adaptive from the viewpoint of LA's inhabitants ☺
- The Heart dynamics
  - Very adaptive: fast response to stimuli (pump more blood on need), robust behavior (restore its normal flow a few minutes later)
- The Heart structure
  - Non adaptive: once damaged, it cannot repair itself (e.g., think at heart attacks!)

Adaptive Individuals vs. Adaptive Systems

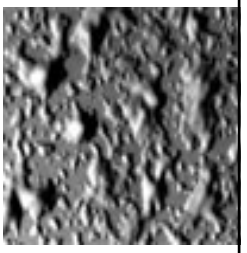
- Sometimes, individual may be adaptive
  - Cats can live anywhere and adapt their food habits accordingly
  - Chameleons change color depending on the environment
- Making the whole system consequently adaptive
  - Cat colonies live and prosper anywhere on the planet
- But sometimes individual capabilities the system overall may not
  - Chameleons communities strongly endangered by pollution and (guess what) by cats ☹
  - The "dynamics" of the chameleons system outperforms the dynamics of the individuals





Non Adaptive Individual vs. Adaptive Systems

- In several cases, ensembles of very simple, purely "reactive" components, may overall lead to very adaptive systems
  - The cell of the heart are not per se adaptive or "smart", nevertheless...
  - The simple unicellular components of the Dictyostelium group together to hunt
- In these cases, adaptivity is a capability that "emerges" from the system and from its interactions
  - Induced by the specific dynamical behavior of the system, as determines by the simple behavior of for the very fact of being a system with a specific dynamics
- Here, adaptivity is a consequence of complexity…



Adaptive Computing Systems

- Let us now turn our attention to various computational systems, and see
  - why they have to be adaptive
  - how they can achieve adaptivity

Adaptive Operating Systems

- Let us consider a Win PC
- Opennes
  - New programs (setup Wizards)
    - Adapt registry and bars (but Win also adapt to Fake programs, unfortunately…)
    - Install necessary dll to enable interactions with new programs
    - And have all programs start working in a concertated way (not always, unfortunately…)
  - New hardware (Plug & Play)
    - Adapt drivers
- Environmental Dynamics
  - If the environment of a PC is its file systems, Win does not adapt to perturbations or dynamics in it

Adaptive Web Server

- Openness
  - New services
    - Enable new services to be added at any time (as in tomcat)
    - Enable services to get access to the server environment and to detect its properties (e.g., the servlet context)
    - Enable services to interact with each other (via Servlet attributes)
  - New Clients
    - Accept connections from new clients
    - Possibly, store clients profile
- Environmental Dynamics
  - The environment of a Web server can be considered as the set of all resources (DBMS, files, pictures, etc) it can access
  - And a service can be programmed to react to contingencies in the environment

The Web

- The ensemble of the Web resources
- Openness
  - Clearly, any new server and Web pages can be added…
    - But this does not adaptively enter the Web world (i.e., link between pages does not automatically create)
    - Nor clients automatically discover the presence of
    - Only major search engines are adaptive, at the price of periodically polling the whole web…
  - Web pages and site can disappear
    - But link to these pages does not automatically adapt…
- Environmental Dynamics
  - Only changes in the content of Web pages can be properly tolerated…

Distributed Applications

- Consider e.g., the case of a Java application distributed over multiple computer
  - Openness
  - New objects can be created on one site
  - And they need to be invoked by other sites
  - Requires mechanisms (e.g., RMI registry) to enable these interactions
- In general, adaptive distributed computing require proper software, to complement current functionalities of servers and of operating systems
  - Enabling distributed component to interact in an open world
  - Enabling distributed components to understand what is happening in the environment
- This is "middleware" → subject of the next lessons

P2P Networks

- E.g., Gnutella, Kazaa, e-Donkey…
  - Openness
  - Any new member can install the program and connect to the network
  - Any member can detach at any time
- Environmental Dynamics
  - Any new file can be shared at any time
- Actually, P2P networks are the only really deployed use of adaptive (and complex, as we will see) system
  - Paradigmatic example
  - With an importance well beyond the current "video-music sharing" mainstream use
- Will get back to these next in the course…

Computational Openness and Network Dynamics

- Openness is a general characteristics of modern distributed systems
  - **Decentralization**: anyone can add computers and components (Web pages and components) in the Internet. No central control.
  - **Mobility**: users move while being connected to the network (PDA and mobile phones) so that the structure of the system continously change
- Also, consumer and embedded computing systems
  - **Ephemeral**: Lead overall to very dynamic systems, with nodes coming on and away at any time

Situatedness in Physical Environments

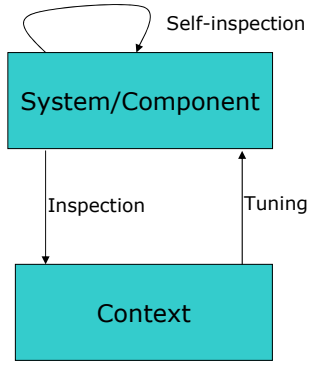
- Modern computing systems are more and more made able to interact with the physical world
  - **Localization mechanism**, GPS
  - **Sensors and actuators** to understand and affect what is happening in the physical world
- Therefore, the physical world is becoming a central part in distributed computing
  - Location aware services, that adapt their behavior depending on where the user is
    - A GPRS service that tells you about the closest restaurants in a city…
  - Situation-aware services:
    - a smart screen that adapt its lightening depending on the external visibility
    - A security sensor that recognize illegal activities in a park and alert the police
  - More generally, "ambient intelligent" systems

Context-awareness

- More in general, for computing to be adaptive it has to be
  - **Context-Aware**
- Which implies
  - The system or its components must be able to recognize the context in which they situates
    - A computational context (the set of file, services, communication and computation resources, as e.g., in servlet context)
    - A physical context (the real-world)
  - Must recognize changes in context
  - Must adapt their behavior to changes in such context
- This can be achieved either via the adaptivity capability of the individuals, or via the adaptive capability of the system as a whole
- So, it is rather clear that adaptivity may be strictly related to the overall dynamics of a system…

Adaptivity via Context-awareness

- In general, it is achieved via
  - Inspection of the context
  - Tuning of behavior depending on the context
- This can occur at the level of individual components or at the level of the system
  - And it can include "self-inspection"
  - the systems and its component are by definition part of the context
- It can be supported by proper technologies to help
- Recognizing context and properties of the system
- "middleware"



Part 3

- What is a Complex System?

What is a Complex System?

- A system is complex when it is hard to infer its dynamics from the behavior and state of its individual components → difficult or impossible to detect its evolution in the phase space
- There are various reasons for this:
  - **Local interactions**: usually, interactions occur among a limited set of "neighbor" components, so that their global transitive effect on the global system state if hard to be elaborated
  - **Non linear dynamics**: e.g., x(t+1) = x(t)(1-x(t))
  - And **feedbacks in interactions**: A→B, B→A,
    - Both typically leading to equations difficult to solve, and to complex phase space topologies with multiple complex attractors. For instance, a system can evolve in very differentiated ways even from very close initial points in the phase space ("butterfly effect").
  - **Openness and Environmental Dynamics**: these prevent
    - A definite modeling of the system, as the system keeps on changing
    - Any predictive modeling of the behavior. The system is always kept out of equilibrium, so that it is difficult to simply trying to see how a system will evolve in the phase state diagram. Probabilistic models must be considered

Complex vs. Complicated Systems

- Clearly, in nature and engineering there are many systems which are very complicated
  - Many components, many interactions
- However, a system which is only complicated and not complex typically has
  - Linear interactions, facilitating modeling and integration of dynamical equations
  - No feedback loops, avoiding the butterfly effect
  - A single or a limited set of stable equilibrium points or limit cycles
  - Contiguous points in the phase space attracts towards the same attractors
  - It is closed, or has well defined boundaries with predictable interactions across boundaries. So it can be treated determi
- Example:
  - The engine of a car can be very complicated, but it is not complex
    - A turbo engine can become complex (the feedback cycle induced by the turbo injector)
    - A badly cooled system can become complex, due to the thermodynamic interactions with the external world
  - A microprocessor, even with billion transistors, it typically only "complicated" → very linear and clean transition rules

Complex Distributed Computational Systems

- Most modern distributed computational systems are indeed complex
- They are open and situated, which is per se an important driving force for complexity
  - We have analyzed this w.r.t. adaptivity
- In addition, they are often based on local interactions
  - E.g., P2P system, mobile systems
- Interactions are not linear
  - E.g., in the Web, the time to get a service grows more than linearly with the number of concurrent requests
  - Effects such as TCP timeouts introduce non-linearity
  - The behavior of humans is greatly non-linear, necessarily reflecting the network system
- And they contain feedback loops
  - The network has loops
  - The more we wait for a service, the more we keep on polling (reinforcement feedback)
  - In P2P network, the more the users the more the system grows and get used

Complexity vs. Size

- Size (the number of components in a system) per se, is not a key reason for complexity
  - There are several huge systems which are simple and linear
  - Control hierarchies, engines, microprocessors
- However, size may be a necessary pre-condition for complexity
  - E.g., Two gravitational masses cannot be a complex system, Three masses are (the three body problem), Asteroids in the Kuiper belt are subject to complex movements
- For Distributed Computational Systems
  - The size may be so large that complexity can hide in them
  - In general, size (together with decentralization) prevents a complete micro-level knowledge of its local components state, e.g., it is nearly impossible to know exactly all the peers in Kazaa

Complexity vs. Chaos

- It is important to note that a complex system it not necessarily chaotic
- Rather, many types of complex systems found in nature are at the "edge" of order and chaos: critical order
  - **Order**: global equilibrium of the system, uniformity
  - **Chaos**: disordered out-of-equilibrium behavior, unorganized non-uniformity
  - **Edges**: out-of-equilibrium, organized and structured behavior, although not exactly predictable
- Clearly, changing some working parameter of the system may lead it to become ordered or chaotic…
- Please note that the "chaos" word often refer to system that are subject to the butterfly effect (e.g., the Earth atmosphere).
  - However, the butterfly effect does not imply that the system is chaotic, but simply that
  - It can evolve in very different configurations starting from however close initial conditions

Complexity vs. Chaos: Examples

- A Dead Heart
  - Static equilibrium, total order
- A Health Heart
  - Global synchronization pattern (order!) of its cells, achieved via local interactions and feedbacks
  - However, the synch is never so exact, and may slightly vary in period
- A Heart during an attack
  - Fibrillation, chaotic pulsing of its cells

Self-organization in Nature

- Many natural systems – even when composed of very simple or stupid elements, appears able to self-organize themselves

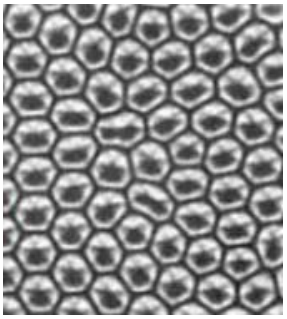
  "Self-organization is a process in which pattern at the global level of a system emerges solely from numerous interactions among the lower level components of the system". [Deneubourg 1977]

- Actually, these systems exhibit a behavior which is clearly at the edge between order and chaos
- In most cases, due to the presence of non-linearity and both positive and negative feedbacks
  - Positive feedbacks introduce amplifications and move the system out of equilibrium
  - Negative feedbacks self-regulate the system and avoid it to become chaotic
- The behavior is considered "emergent"

Examples of Self-organization in Nature (1)

- Patterns in Physics



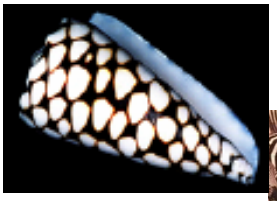




Examples of Self-organization in Nature (2)

- Patterns in Skins and Shells

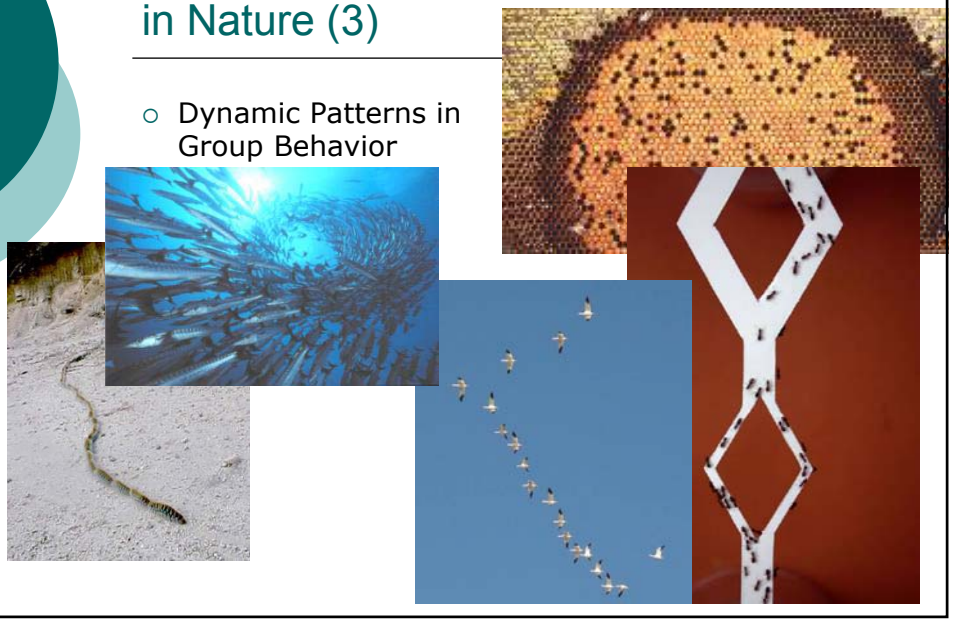







Examples of Self-organization in Nature (3)

- Dynamic Patterns in Group Behavior



Examples of Self-organization in Nature (4)

- Emergent Behaviors in Societies
  - Clapping synchronization
  - Mexican Wave ("La Ola")
  - Patterns of Walking in crowdy pathways
  - Patterns of epidemics
  - Patterns of acquaintances

Complexity vs. Adaptivity

- A self-organized system may be adaptive for a wide range of environmental perturbations
  - It can re-establish global patterns upon changed conditions
  - Typically not the same, but similar
  - It moves from an attractor to an equivalent one
  - The presence of negative feedbacks, provide for re-establishing order
  - Clearly, excessive perturbations may disrupt the system or move it to chaotic state
    - E.g., the human heart
- Also, openness is a limited problem
  - New components can get in the system
  - Without affecting significantly its overall dynamics

Why Self Organization in Distributed Systems?

- In large, decentralized, situated and dynamics networks
  - High complexity of software and underlying hardware
    - very hard to be managed and engineered, due to size and non linear interactions
  - Continuous updated and tuning needed
    - Dynamic changes in the network and in the system (openness)
    - Dynamic changes in the environment
  - Impossibility of direct control and configuration
    - E.g., cannot control all the nodes of the Internet or of a P2P network
    - Cannot control and access all the sensors and embedded computers distributed in an environment
  - Impossibility of direct maintenance
    - Cannot re-configure manually all the nodes
    - Application must not stop working and should preserve specific quality levels

Why Self Organization in Distributed Systems? Also…

- Commercial and Practical Reasons
- And even if we could control the configuration and the adaptation/maintenance of complex systems that would be
  - Economically unfeasible
    - Too high development and maintenance costs
  - Commercially unacceptable
    - Who wants a system which is always in need of configuration?
- Computing systems must become
  - Proactive, self-adapting and self-organizing with "humans out-of-the-loop" (Tennenhouse, 2001)
  - Autonomic, as if they were living organisms (Kephart, 2003)
  - Easy and painless to be used as "sprays" (Zambonelli, 2005)

What is Computational Self-Organization?

- Components/applications get deployed and:
  - They recognize who and where they are (w.r.t. the other components and the environment)
  - They identify their specific task in the network (according to the who and where)
  - They start working in cooperation with the other components to achieve their task
  - The global goal/configuration reached without supervision
  - Achiving in a spontaneous way **emergent, self-organized, coherent behaviors**
- Upon dynamic changes
  - They recognize such changes
  - And re-adapt to suit the new situation

Part 4

- Distributed Computational Systems as CAS

A Vision of the Future…

- ***C***omputing everywhere and at every scale
  - In every room and object
  - At every scale (from nano- and micro-devices, to super computers)
- ***N***etworking everywhere and at every scale
  - Small local micro-sensor networks (networks of small embedded computer systems to sense and interact with our environments)
  - Pervasive computing systems: all our objects and artifact will be computer-based and will network with each other
  - Wired, Wireless, and short-range radio communications
- ***M***obility and *Ephemerality* everywhere at every scale
  - We move with our laptops, PDAs and cell phones
  - Furniture can be moved around the house
  - Small wheeled micro-bots
  - All of which coming and leaving at any time

C.N.M. Everywhere: the Big Scale

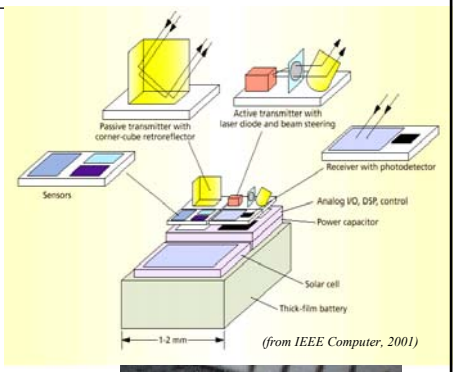
- World-wide networks (different levels)
  - Routers and IP nodes
  - The Web network
  - P2P Networks (e.g., Gnutella, Freenet)
- Millions of interconnected computer-based (or software) components
- Mobility….more and more….
  - Slow mobility (I connect my laptop from different places at different days)
  - Fast mobility (I want to stay connected while roaming…)
  - Ephemerality: components and services coming and leaving…

C.N.M. Everywhere: the Medium Scale

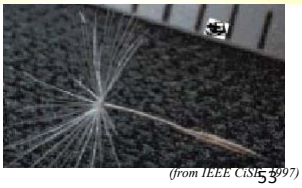
- Computer-based objects
  - PDAs and cell phones
  - Intelligent hardware (e.g., fridges, washing machines, etc.)
  - Smart artifacts (e.g., doors, tables, etc.)
  - Embedded sensors
- Communication enabled
  - E.g., small area networks (Bluetooth), IR, Wi-Fi
- Mobility
  - We move with carried-on objects
  - We move objects
  - Some objects move (e.g., cars)
  - All of these being of an ephemeral nature

C.N.M. Everywhere: the Small Scale (1)

- Micro computer-based devices:
  - E.g., "smart dust"
  - 1mm^3 computer with optical I/O capabilities
  - IR or ultra-wide band short range communications
- Large scale production possible
  - Foreseen price << 1$

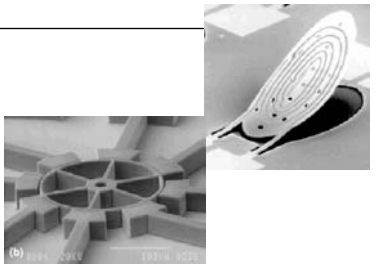


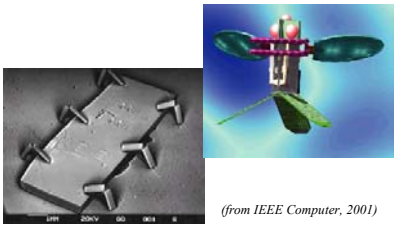
(from IEEE Computer, 2001)


(from IEEE CISE, 1997)

C.N.M. Everywhere: the Small Scale (2)

- MEMS
  - Specific mechanical actions (sensing of movement, pressure,)
  - Possibly coupled with limited computing and communication capabilities
- And micro-robots…
  - Mobile!




(from IEEE Computer, 2001)

The Overall Perspective

- All these define complex distributed systems
- Large scale both as isolated network
  - Hundreds to millions of nodes
- And altogether, leading to very huge networks:
  - Billions of interconnected computers, from micro to macro ones ("The embedded Internet")
  - e.g. IPv6 will make it possible to individually address each and every mm^2 on the earth surface….
- Characterized by dramatic dynamics and openness at all scales
  - Nodes and components arriving, leaving, changing location
  - E.g., Internet nodes and Web sites, cell-phones, micro-sensors
- And also at the level of software
  - New applications deployed at any time
  - intrinsic inter-dependencies between different applications
- Typically situated and interacting with computational and physical environments
  - They have to regulate/adapt activities depending on the environmental situation

So What?

- There is need for brand new:
  - Theoretical models
  - Algorithms
  - Middleware infrastructures
  - Programming abstractions
  - Methodologies and tools
- For building distributed computing systems that can
  - Show the same properties of natural complex adaptive systems
  - Rely on self-organization and emergent organization
  - Be robust to dynamics and be highly adaptive
- A great commercial advantage could be achieved
- And a number of potential innovative applications could be conceived and deployed….

Application Visions….

- The self-organizing Web (that's already on the way…)
  - P2P access to data and services
  - Dynamic and self-healing re-structuring of links
  - Tolerating, e.g., mobility and faults
  - Relying on self-localization on virtual overlay network structures (e.g., Gnutella links)
- And more:
  - Emergence of communication languages and conventions
  - Emergence of peculiar highly optimized structures (e.g., small worlds and scale free)

Application Visions….

- Spray television
  - Micro computer-based emitters
  - To display TV programs and PC screens
  - Requires:
    - Self localization of components, and possibly distributed time synchronization
- Smart paintings
  - To display colors and various patterns on demand
  - Requires
    - Self localization of components, coordinated emergent behaviors

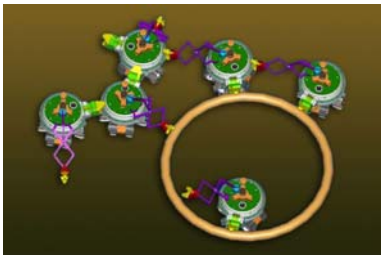
Application Visions….

- Active cereal boxes (remember "Minority Report"?)
  - Painted with optical micro-computers
  - Activated upon movement
  - Start animating a cartoon
  - Requires:
    - Self localization, time synchronization, emergent coordinated behaviors, self-differentiation
- Active pipelines
  - Micro components affecting fluid flow
  - And avoiding e.g., turbulence
  - Requires
    - Coordinated distributed sensing and coordinated movements



Application Visions….

- Robot swarms
  - Coordinating their movements in an environment
  - Achieving complex task
  - In a fully autonomous way
- Self-assembly
  - Micro and nano-scale components
  - Capable of orchestrating their movement
  - So as to assume specific shapes
  - Adaptive and self-healing





The Chricton's Vision in "Prey"

- "Swarms" of nano component
  - Mixed organic and silicon-based material
  - Capable of flying by attaching to air molecules
  - Capable of self-assembly
  - Capable of collective self-organizing vision
  - Capable of collective sentient behavior
- Built by a complex self-organized systems of other micro-components
  - The "assemblers"
  - Capable of self-reproduction
- Is this really science fiction?

To Conclude: A Rationale for the Course

- In the following lectures we will:
- Understand the *technologies* to promote adaptivity in not-very-complex systems
- Make *examples* of complex self-organizing computing systems
- See how the lessons of *natural* complex adaptive systems can be apply to the engineering of *computational* complex adaptive
- Discuss some potential innovative *applications*
- Clearly, this also implies that we will *understand* how several natural systems work…

Readings

Articles (read at least one at your choice)

- D. Tennenhouse, "Proactive Computing", Communications of the ACM, May 2001
- J. Kephart, D. Chess, "The Vision of Autonomic Computing", IEEE Computer, January 2003
- F. Zambonelli, V. Parunak, "Towards a Paradigm Change in Computer Science and Software Engineering", Knowledge Engineering Review, September 2004
- F. Zambonelli, M.P. Gleizes, M. Mamei, R. Tolksdorf, "Spray Computers: Explorations in Self-Organization", Pervasive and Mobile Computing, May 2005

Books (strongly suggested to read at least one) ALSO IN ITALIAN

- Mitchell Resnick, Turtles, Termites, and Traffic Jams, MIT Press, 1999
- Steven Johnson, Emergence, 2001.